

Alright, here's your **complete Zero to Expert Web Development Roadmap** for the AI era.
This is comprehensive, detailed, and practical — copy it and follow it step by step. 👉

Complete Web Development Roadmap 2026 — Zero to Expert (AI Era Edition)

Goal: Become a job-ready Full Stack Web Developer

Estimated Timeline: 6–9 months (3–5 hours/day)

Stack Focus: MERN + Next.js (most in-demand in 2026)

Philosophy: Learn concepts deeply → Use AI to accelerate → Build real projects

Golden Rules (Read Before Starting)

1. **Never copy-paste AI code blindly** — always understand what it does before using it.
 2. **Learn concepts first, syntax second** — AI can write syntax, but YOU must understand the logic.
 3. **Build projects at every phase** — tutorials alone won't make you a developer.
 4. **Use AI as a learning partner, not a crutch** — ask it to explain, not just to write.
 5. **Consistency > Speed** — 2 hours daily beats 10 hours once a week.
-

Tools to Set Up Before Starting

Tool	Purpose	Note
VS Code	Code editor	Install extensions: Prettier, ESLint, Live Server, GitLens
Google Chrome	Browser + DevTools	Learn to use Inspect Element
Git	Version control	Install from git-scm.com
GitHub	Code hosting & portfolio	Create an account
Node.js	JavaScript runtime	Install LTS version from nodejs.org
GitHub Copilot	AI coding assistant	Free for students via GitHub Education
ChatGPT / Copilot Chat	AI learning partner	Use for explanations, debugging, code review

¹ ²

¹<https://biteofcode.com/web-dev-roadmap-2026/>

²<https://www.coursera.org/resources/web-development-learning-roadmap>

PHASE 1: The Foundation (Week 1–6)

 **This is the MOST important phase. Do NOT rush it.**

AI Rule: Learn these topics YOURSELF first. Use AI only to explain confusing concepts — NOT to write code for you yet.

1.1 How the Internet Works (Week 1, Day 1–2)

What to learn:

- How browsers work (request → server → response → render)
- HTTP/HTTPS protocol basics (GET, POST, PUT, DELETE)
- What is a URL, domain, DNS, IP address
- Client vs Server model
- What is an API (conceptually)

How to learn:

- Watch: "How the Internet Works" (freeCodeCamp YouTube)
- Read: MDN Web Docs — "How the web works"

AI Usage:

- Ask: "Explain HTTP request-response cycle like I'm 10 years old"
 - Ask: "What happens step by step when I type google.com in my browser?"
-

1.2 HTML5 — Structure of the Web (Week 1–2)

What to learn:

- HTML document structure (<!DOCTYPE>, <html>, <head>, <body>)
- Semantic tags: <header>, <nav>, <main>, <section>, <article>, <footer>
- Text tags: <h1>–<h6>, <p>, , ,
- Links (<a>), Images (), Lists (, ,)
- Tables (<table>, <tr>, <td>, <th>)
- Forms: <form>, <input>, <textarea>, <select>, <button>
- Input types: text, email, password, number, date, checkbox, radio
- Form validation attributes: required, minlength, pattern
- HTML5 APIs awareness: <audio>, <video>, <canvas>
- Accessibility basics: alt text, aria-label, role

Where to learn:

- freeCodeCamp (Responsive Web Design certification)
- MDN Web Docs — HTML reference



AI Usage:

- After building a page manually, ask AI: *"Review my HTML and suggest semantic improvements"*
- Ask: *"What's the difference between <div> and <section>?"*



Mini Project: Build your personal bio page with proper semantic HTML, a form, and a table.

3 4

1.3 CSS3 — Styling & Layout (Week 2–4)

What to learn:

Basics:

- Selectors (element, class, id, attribute, pseudo-class, pseudo-element)
- Box Model (content, padding, border, margin)
- Colors, backgrounds, gradients
- Typography (font-family, font-size, line-height, Google Fonts)
- Units: px, em, rem, %, vh, vw

Layout (CRITICAL):

- Display: block, inline, inline-block, none
- Position: static, relative, absolute, fixed, sticky
- **Flexbox** (must master): flex-direction, justify-content, align-items, flex-wrap, gap
- **CSS Grid** (must master): grid-template-columns/rows, grid-area, gap, auto-fit/auto-fill
- Float (just know it exists, don't focus)

Responsive Design:

- Media queries (@media)

³<https://w3htmlschool.com/blog/web-development-roadmap-2026-step-by-step-guide-for-beginners/>

⁴<https://www.geeksforgeeks.org/blogs/full-stack-developer-roadmap/>

- Mobile-first approach
- Responsive images
- Viewport meta tag

Animations & Transitions:

- transition property
- transform (translate, rotate, scale)
- @keyframes animation
- hover, focus effects

Where to learn:

- freeCodeCamp CSS section
- Kevin Powell YouTube channel (best CSS teacher)
- CSS Tricks (Flexbox guide, Grid guide)



AI Usage:

- Build layouts yourself first, then ask AI: *"How can I center this div vertically and horizontally?"*
- Ask: *"Convert this desktop layout to be responsive for mobile"*
- Ask: *"Explain CSS specificity with examples"*



Mini Projects:

1. Clone a simple landing page (e.g., a restaurant website)
2. Build a responsive portfolio page with Flexbox/Grid

5 6

1.4 JavaScript — The Brain of the Web (Week 4–8)



This is THE most important language you'll learn. Spend serious time here.

What to learn:

Basics (Week 4–5):

- Variables: let, const, var (and why var is outdated)
- Data types: string, number, boolean, null, undefined, object, array
- Operators: arithmetic, comparison, logical, ternary

⁵<https://biteofcode.com/web-dev-roadmap-2026/>

⁶<https://dev.to/codersnote/the-complete-full-stack-roadmap-2026-100-free-resources-l3e>

- Type conversion & coercion
- Strings: template literals, string methods
- Conditionals: if/else, switch
- Loops: for, while, for...of, for...in
- Functions: declaration, expression, arrow functions, parameters, return
- Scope: global, function, block scope
- Hoisting (conceptually)

Intermediate (Week 5–6):

- Arrays: methods — map(), filter(), reduce(), find(), forEach(), sort(), slice(), splice()
- Objects: properties, methods, destructuring, spread operator ...
- Array/Object destructuring
- Rest parameters
- Modules: import / export
- Error handling: try/catch/finally
- JSON: JSON.parse(), JSON.stringify()

DOM Manipulation (Week 6–7):

- What is the DOM (Document Object Model)
- Selecting elements: getElementById, querySelector, querySelectorAll
- Modifying elements: textContent, innerHTML, style, classList
- Creating/removing elements: createElement, appendChild, remove()
- Events: addEventListener, event types (click, submit, keydown, input)
- Event delegation, event bubbling
- Form handling with JS

Advanced (Week 7–8):

- Closures
- this keyword
- Higher-order functions
- Callbacks
- Promises: new Promise(), .then(), .catch(), .finally()
- async / await
- Fetch API: fetch() for API calls
- setTimeout, setInterval
- LocalStorage & SessionStorage
- Basic Regular Expressions

Where to learn:

- JavaScript.info (best free JS resource)

- freeCodeCamp JavaScript section
- "JavaScript: Understanding the Weird Parts" (YouTube)
- Eloquent JavaScript (free online book)



AI Usage:

- Write code yourself first, then ask AI: *"What's wrong with this code?"*
- Ask: *"Explain closures with a real-world analogy"*
- Ask: *"Give me 5 practice problems using array methods"*
- **DO NOT** ask AI to write JS logic for you at this stage — you **MUST** build this muscle



Projects:

1. To-Do List app (DOM manipulation + localStorage)
2. Weather App (Fetch API + external API)
3. Quiz App (logic + DOM + events)
4. Simple Calculator

7 8 9

1.5 Git & GitHub — Version Control (During Week 4–5)

What to learn:

- What is version control and why it matters
- git init, git add, git commit, git status, git log
- git branch, git checkout, git merge
- git push, git pull, git clone
- .gitignore
- Creating repositories on GitHub
- Pull Requests (PR) basics
- Writing good commit messages

Where to learn:

- "Git and GitHub for Beginners" (freeCodeCamp YouTube)
- Atlassian Git tutorials

⁷<https://www.geeksforgeeks.org/blogs/full-stack-developer-roadmap/>

⁸<https://dev.to/developerhint/full-stack-web-developer-roadmap-2026-complete-guide-from-beginner-to-advanced-fam>

⁹<https://talent500.com/blog/full-stack-developer-roadmap-2026/>

AI Usage:

- Ask: *"I have a merge conflict, explain how to resolve it"*
- Ask: *"What's the difference between git merge and git rebase?"*

✅ **Practice:** Push ALL your projects to GitHub from now on. This becomes your portfolio.

10 11

PHASE 2: Frontend Framework (Week 9–14)

AI Rule: Now you can start using AI more — for boilerplate, component structure, and debugging. But always understand what the generated code does.

2.1 TypeScript — Typed JavaScript (Week 9–10)

What to learn:

- Why TypeScript exists (catches bugs before runtime)
- Basic types: string, number, boolean, array, object
- Type annotations & type inference
- Interfaces & Type aliases
- Union types, literal types
- Generics (basics)
- enum
- Type narrowing
- TypeScript with functions (parameter types, return types)

Where to learn:

- TypeScript official handbook (typescriptlang.org)
- Matt Pocock's "Total TypeScript" (YouTube)
- "No BS TS" by Jack Herrington (YouTube)

AI Usage:

- Ask: *"Convert this JavaScript function to TypeScript"*
- Ask: *"What TypeScript type should I use for this API response?"*

¹⁰<https://www.coursera.org/resources/web-development-learning-roadmap>

¹¹<https://dev.to/codersnote/the-complete-full-stack-roadmap-2026-100-free-resources-l3e>

✓ **Practice:** Rewrite one of your Phase 1 JS projects in TypeScript.

12 13

2.2 React.js — UI Library (Week 10–13)

What to learn:

Core Concepts:

- What is React & why use it (component-based, declarative)
- JSX syntax
- Components: functional components (class components are outdated)
- Props: passing data to components
- State: useState hook
- Rendering: conditional rendering, lists & keys
- Event handling in React

Intermediate:

- useEffect hook (side effects, data fetching, cleanup)
- useRef hook
- useContext hook (Context API for global state)
- Custom hooks
- Component lifecycle (conceptually)
- Controlled vs uncontrolled components (forms)
- React Router: routing, nested routes, dynamic routes, useNavigate, useParams

Advanced:

- useReducer hook
- useMemo, useCallback (performance optimization)
- React.memo
- Error Boundaries
- Lazy loading & Suspense
- State management options: Context API vs Zustand vs Redux Toolkit (learn Zustand — it's simpler and modern)

Where to learn:

¹²<https://www.geeksforgeeks.org/blogs/full-stack-developer-roadmap/>

¹³<https://stacksfinder.com/guides/tech-stack-2026>

- React official docs (react.dev) — excellent new docs
- Scrimba "Learn React" course (free)
- "Full Modern React Tutorial" by Net Ninja (YouTube)

AI Usage:

- Ask: *"Generate a basic React component for a product card"* — then modify it yourself
- Ask: *"Explain the useEffect dependency array"*
- Ask: *"Review my React component for best practices"*
- Use GitHub Copilot to autocomplete repetitive JSX

Projects:

1. Movie Search App (React + API)
2. E-commerce Product Listing (with cart using state management)
3. Blog App frontend (React Router + dynamic routes)

14 15

2.3 Tailwind CSS — Utility-First Styling (Week 13–14)

What to learn:

- Utility classes approach vs traditional CSS
- Layout utilities (flex, grid, padding, margin)
- Responsive design with Tailwind (sm:, md:, lg:, xl:)
- Dark mode
- Custom configuration (tailwind.config.js)
- Component patterns with Tailwind
- Tailwind + React integration

Where to learn:

- Tailwind CSS official docs (tailwindcss.com)
- "Tailwind CSS Full Course" by Dave Gray (YouTube)

AI Usage:

¹⁴<https://scrimba.com/articles/how-to-become-a-full-stack-developer-in-2026-complete-roadmap/>

¹⁵<https://dev.to/developerhint/full-stack-web-developer-roadmap-2026-complete-guide-from-beginner-to-advanced-fam>

- Ask: *"Convert this CSS to Tailwind classes"*
- Ask: *"Create a responsive navbar using Tailwind CSS"*
- Copilot is VERY good at Tailwind — let it autocomplete classes

✓ **Project:** Rebuild your portfolio website using React + Tailwind CSS

16

PHASE 3: Backend Development (Week 15–22)

AI Rule: Use AI freely for boilerplate and repetitive backend code. Focus YOUR energy on understanding architecture, security, and data flow.

3.1 Node.js — JavaScript Runtime (Week 15–16)

What to learn:

- What is Node.js (JS outside the browser)
- Node.js modules: fs, path, http, os
- NPM (Node Package Manager): npm init, npm install, package.json
- CommonJS (require) vs ES Modules (import)
- Environment variables (.env files, dotenv)
- Event loop (conceptually — very important)
- Streams & Buffers (basic awareness)

Where to learn:

- Node.js official docs
 - "Node.js Tutorial for Beginners" by Programming with Mosh (YouTube)
-

3.2 Express.js — Backend Framework (Week 16–18)

What to learn:

- Setting up an Express server
- Routing: GET, POST, PUT, DELETE
- Middleware concept (very important)
- Request & Response objects (req.body, req.params, req.query)
- Error handling middleware

¹⁶<https://learn.programming-hero.com/web-development-roadmap-2026/>

- CORS (Cross-Origin Resource Sharing)
- File uploads (Multer)
- Serving static files
- Environment-based configuration
- MVC pattern (Model-View-Controller)

AI Usage:

- Ask: *"Generate Express boilerplate with error handling middleware"*
- Ask: *"Explain middleware chain execution order"*
- Use AI to generate repetitive CRUD routes, then customize them

 **Project:** Build a REST API for a blog (CRUD operations for posts, users)

17 18

3.3 Databases (Week 18–20)

MongoDB (NoSQL) — Primary

What to learn:

- What is NoSQL vs SQL
- MongoDB basics: documents, collections, databases
- CRUD operations in MongoDB
- Mongoose ODM (schemas, models, validation)
- Relationships: embedding vs referencing
- Indexing basics
- Aggregation pipeline (basics)

PostgreSQL (SQL) — Secondary (learn basics)

What to learn:

- SQL fundamentals: SELECT, INSERT, UPDATE, DELETE
- JOINS: INNER, LEFT, RIGHT
- Tables, primary keys, foreign keys
- Relationships: one-to-one, one-to-many, many-to-many

¹⁷<https://dev.to/developerhint/full-stack-web-developer-roadmap-2026-complete-guide-from-beginner-to-advanced-fam>

¹⁸<https://scrimba.com/articles/how-to-become-a-full-stack-developer-in-2026-complete-roadmap/>

- Basic normalization

Where to learn:

- MongoDB University (free courses)
- "SQL Tutorial" by freeCodeCamp (YouTube)
- Prisma docs (for PostgreSQL ORM later)

AI Usage:

- Ask: *"Design a MongoDB schema for an e-commerce app"*
 - Ask: *"Write a SQL query to get all orders with customer names"*
 - Ask: *"Should I use embedding or referencing for this relationship?"*
-

3.4 Authentication & Authorization (Week 20–21)

What to learn:

- Password hashing: bcrypt
- JWT (JSON Web Tokens): what they are, how they work
- Login/Register flow implementation
- Protecting routes (auth middleware)
- Refresh tokens
- OAuth basics (Google login, GitHub login)
- Session vs Token-based auth
- Role-based access control (admin, user)

AI Usage:

- Ask: *"Generate JWT authentication middleware for Express"*
 - Ask: *"Explain the security risks of storing JWT in localStorage vs cookies"*
-

3.5 REST API Design (Week 21–22)

What to learn:

- RESTful conventions (resource naming, HTTP methods)
- Status codes: 200, 201, 400, 401, 403, 404, 500
- Request validation (Joi or Zod)
- Pagination, filtering, sorting
- Rate limiting
- API documentation (Swagger/OpenAPI basics)
- Error response format standardization

✓ **BIG PROJECT:** Build a complete backend for a Task Management App with:

- User registration/login (JWT)
- CRUD for tasks
- Role-based access
- Pagination
- Input validation

19 20

🌀 PHASE 4: Full Stack Integration + Next.js (Week 23–30)

AI Rule: Now use AI as a productivity multiplier. Generate boilerplate, debug faster, write tests. But YOU make all architecture decisions.

4.1 Next.js — The Full Stack React Framework (Week 23–27)

What to learn:

Core:

- What is Next.js and why use it (SSR, SSG, ISR, API routes)
- App Router (new) vs Pages Router (legacy — know it exists)
- File-based routing
- Server Components vs Client Components ("use client")
- Layouts and nested layouts
- Loading & error states
- <Link>, <Image> components

Data Fetching:

- Server-side data fetching (in Server Components)
- fetch() with caching options
- Server Actions (form handling without API routes)
- Dynamic routes with [params]

API Routes:

¹⁹<https://talent500.com/blog/full-stack-developer-roadmap-2026/>

²⁰<https://www.linkedin.com/pulse/modern-full-stack-roadmap-2026-what-learn-ignore-sandeep-pal-kugtc>

- Route Handlers (app/api/...)
- Building APIs within Next.js

Advanced:

- Middleware (authentication, redirects)
- Static Site Generation (SSG) vs Server-Side Rendering (SSR)
- Incremental Static Regeneration (ISR)
- SEO: metadata API, generateMetadata
- Deployment on **Vercel** (one-click deploy)

Where to learn:

- Next.js official docs (nextjs.org/learn) — excellent interactive tutorial
- "Next.js 14 Full Course" by JavaScript Mastery (YouTube)
- Vercel's official YouTube channel



AI Usage:

- Ask: *"Should this be a Server Component or Client Component?"*
- Ask: *"Generate a Next.js API route for user authentication"*
- Ask: *"Optimize this Next.js page for SEO"*

21 22

4.2 Database as a Service — Supabase or Firebase (Week 27–28)

Since you already use **Supabase**, lean into it!

What to learn:

- Supabase setup (project creation, tables)
- Supabase Auth (email/password, OAuth providers)
- Supabase Database (PostgreSQL underneath)
- Row Level Security (RLS)
- Supabase Storage (file uploads)
- Real-time subscriptions
- Supabase + Next.js integration

²¹<https://scrimba.com/articles/how-to-become-a-full-stack-developer-in-2026-complete-roadmap/>

²²<https://www.aitechworlds.com/2026/04/web-development-roadmap-2026-complete.html>

Alternative: Firebase (if you want to explore Google's ecosystem)

 AI Usage:

- Ask: *"Set up Supabase auth with Next.js App Router"*
 - Ask: *"Write a Row Level Security policy for this table"*
-

4.3 ORM — Prisma (Week 28–29)

What to learn:

- What is an ORM and why use one
- Prisma schema definition
- Migrations: `prisma migrate dev`
- CRUD operations with Prisma Client
- Relations in Prisma
- Prisma + Next.js integration
- Prisma + PostgreSQL / Supabase

 AI Usage:

- Ask: *"Generate a Prisma schema for a blog with users, posts, and comments"*
 - Ask: *"Write a Prisma query with pagination and filtering"*
-

4.4 Deployment & DevOps Basics (Week 29–30)

What to learn:

- **Vercel** — deploy Next.js (primary choice)
- **Netlify** — deploy static sites
- **Railway** or **Render** — deploy Node.js/Express backends
- Environment variables in production
- Domain setup (custom domains)
- CI/CD basics (GitHub Actions — basic workflow)
- Docker basics (just awareness: what containers are, Dockerfile basics)
- Linux basics (you already know Ubuntu! )

 AI Usage:

- Ask: *"Write a GitHub Actions workflow to deploy on push to main"*
- Ask: *"Create a Dockerfile for my Node.js app"*

 **MEGA PROJECT:** Build and deploy a **Full Stack Application** such as:

-  Project Management Tool (like Trello clone)
-  E-commerce Store
-  Blog Platform with CMS
-  Online Learning Platform

Stack: Next.js + Supabase/Prisma + Tailwind CSS + Vercel

23 24

PHASE 5: Advanced & Expert Level (Week 31–40+)

AI Rule: At this level, AI is your co-developer. You architect, AI implements the boring parts, you review and refine.

5.1 Testing (Week 31–32)

What to learn:

- Why testing matters
 - Unit testing with **Jest**
 - Component testing with **React Testing Library**
 - Integration testing
 - End-to-end testing with **Playwright** or **Cypress**
 - Test-driven development (TDD) basics
-

5.2 Performance Optimization (Week 32–33)

What to learn:

- Core Web Vitals (LCP, FID, CLS)
- Lighthouse audits
- Image optimization
- Code splitting & lazy loading
- Caching strategies
- Database query optimization

²³<https://stacksfinder.com/guides/tech-stack-2026>

²⁴<https://scrimba.com/articles/how-to-become-a-full-stack-developer-in-2026-complete-roadmap/>

- CDN basics
-

5.3 System Design & Architecture (Week 33–35)

What to learn:

- Monolith vs Microservices (conceptually)
 - API design patterns
 - Caching (Redis basics)
 - Message queues (concept)
 - WebSockets (real-time communication)
 - Rate limiting, load balancing (concept)
 - Database scaling (concept: horizontal vs vertical)
-

5.4 AI Integration Skills (Week 35–37)

 **This is what makes you a 2026 developer**

What to learn:

- OpenAI API / Anthropic API integration
- Building AI-powered features in your apps
- Prompt engineering for developers
- LangChain basics (for chaining AI calls)
- Embedding AI chatbots in web apps
- AI-assisted code review workflows
- Using MCP (Model Context Protocol) with dev tools

AI Usage:

- Build an AI chatbot feature in your project
- Use AI to generate and review code in your workflow
- Ask: *"How do I integrate OpenAI API into my Next.js app?"*

25 26

²⁵https://blog.weskill.org/2026/04/the-2026-web-developer-roadmap-future_0791659955.html

²⁶<https://www.aitechworlds.com/2026/04/web-development-roadmap-2026-complete.html>

5.5 Soft Skills & Career Prep (Ongoing)

What to learn:

- Technical writing (README, documentation)
- Communication skills
- Problem-solving approach (how to think before coding)
- Code review etiquette
- Agile/Scrum basics
- Resume building
- Portfolio website (showcase 4–5 best projects)
- LinkedIn optimization
- Open source contribution (start small)
- DSA basics for interviews (LeetCode Easy/Medium)

 Complete Technology Summary Table

Category	Technologies	Priority
Languages	HTML, CSS, JavaScript, TypeScript, SQL	Must
Frontend Framework	React.js	Must
CSS Framework	Tailwind CSS	Must
Full Stack Framework	Next.js (App Router)	Must
Backend	Node.js, Express.js	Must
Database (NoSQL)	MongoDB + Mongoose	Important
Database (SQL)	PostgreSQL + Prisma	Must
BaaS	Supabase	Important
Auth	JWT, bcrypt, OAuth, Supabase Auth	Must
Version Control	Git, GitHub	Must
Deployment	Vercel, Netlify, Railway	Must
Testing	Jest, React Testing Library, Playwright	Important
State Management	Zustand (or Redux Toolkit)	Important
API Tools	Postman, Swagger	Important
AI Tools	GitHub Copilot, ChatGPT/Claude, Cursor IDE	Must
Containerization	Docker (basics)	Nice to have
CI/CD	GitHub Actions	Nice to have

 How to Use AI at Each Stage — Summary

Phase	AI Usage Level	What AI Does	What YOU Do
-------	----------------	--------------	-------------

Phase 1 (Foundation)	● Low (20%)	Explains concepts, answers doubts	Write ALL code yourself
Phase 2 (Frontend)	● Medium (40%)	Generates component boilerplate, reviews code	Understand architecture, build logic
Phase 3 (Backend)	● Medium (50%)	Generates CRUD routes, schemas, middleware	Design API structure, handle security
Phase 4 (Full Stack)	● High (60%)	Generates boilerplate, configs, deployment scripts	Make architecture decisions, integrate systems
Phase 5 (Expert)	● Full Partner (70%)	Co-develops features, writes tests, reviews	Architect, verify, optimize, lead

27 28

Free Learning Resources (Best of the Best)

Resource	What For	Link
freeCodeCamp	HTML, CSS, JS, React	freecodecamp.org
The Odin Project	Full stack path	theodinproject.com
JavaScript.info	Deep JS learning	javascript.info
MDN Web Docs	Reference for everything	developer.mozilla.org
React Official Docs	React learning	react.dev
Next.js Learn	Next.js tutorial	nextjs.org/learn
roadmap.sh	Visual roadmaps	roadmap.sh
Scrimba	Interactive courses	scrimba.com
LeetCode	DSA practice	leetcode.com
YouTube Channels	Tutorials	Traversy Media, Net Ninja, Fireship, Kevin Powell, Web Dev Simplified

29 30

²⁷<https://www.aitechworlds.com/2026/04/web-development-roadmap-2026-complete.html>

²⁸<https://biteofcode.com/web-dev-roadmap-2026/>

²⁹<https://roadmap.sh/>

³⁰<https://dev.to/codersnote/the-complete-full-stack-roadmap-2026-100-free-resources-l3e>

Final Portfolio Checklist (Before Applying for Jobs)

- **Personal Portfolio Website** (React/Next.js + Tailwind + deployed)
 - **3–5 Full Stack Projects** on GitHub with clean README
 - **Active GitHub** profile (green contribution graph)
 - **LinkedIn** profile with projects linked
 - **Resume** — 1 page, clean, focused on projects & skills
 - **Basic DSA** — Can solve LeetCode Easy and some Medium problems
 - **Can explain** your projects in an interview (architecture, challenges, decisions)
-

This is your complete map, Marajul. You already have strong technical foundations (Ubuntu, Supabase, Flutter, dev tools) — so you'll move faster than a true beginner. The key difference in 2026 is: **the developer who understands WHY the code works will always beat the developer who only knows HOW to prompt AI to write it.** ^{31 32}

Want me to break down any specific phase in even more detail, or create a **day-by-day schedule** for any particular phase? 🚀

³¹<https://www.aitechworlds.com/2026/04/web-development-roadmap-2026-complete.html>

³²<https://talent500.com/blog/full-stack-developer-roadmap-2026/>